НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

Звіт з виконання лабораторної роботи №3
з дисципліни «Спеціальні розділи обчислювальної математики»

РЕАЛІЗАЦІЯ ОПЕРАЦІЙ У СКІНЧЕННИХ ПОЛЯХ ХАРАКТЕРИСТИКИ 2

Виконала студентка
групи ФІ-43
Рєпко Єлизавета Сергіївна
Варіант 7

Київ — 2026

1 Мета роботи

Одержання практичних навичок програмної реалізації обчислень у полі Галуа характеристики 2 в поліноміальному базисі; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

2 Завдання до комп'ютерного практикуму

Основне завдання: Реалізувати поле Галуа характеристики 2 степеня m в поліноміальному базисі з 9 базовими операціями (константи 0 і 1, додавання, множення, слід, квадрат, піднесення до степеня, обернений елемент та конвертування форматів). Проконтролювати коректність реалізації та визначити середній час виконання операцій.

Варіант 7: Розмірність розширення $m = 251$. Незвідний поліном (генератор поля): $p(x) = x^{251} + x^{14} + x^4 + x + 1$.

3 Теоретичні відомості

У поліноміальному зображенні елементи поля $GF(2^m)$ являють собою многочлени степеня, що не перевищує $m-1$, над базовим полем $GF(2)$. Коефіцієнти цих многочленів набувають значень 0 або 1. Арифметика поля має наступні властивості:

- **Додавання та віднімання** у $GF(2^m)$ є тотожними операціями і виконуються як звичайне додавання поліномів над $GF(2)$, що еквівалентно покомпонентній операції `XOR` двійкових векторів.

- **Множення** виконується як класичне перемноження многочленів з наступним зведенням (редукцією) результату за модулем незвідного многочлена $p(x)$.

- **Піднесення до квадрату** використовує властивість лінійності у полях характеристики 2: $(a+b)^2 = a^2 + b^2$. У бітовому поданні це реалізується через «проріджування» бітового запису (вставляння нулів після кожного біту) з подальшою редукцією.

- **Знаходження оберненого елемента** для $a \neq 0$ спирається на наслідок з теореми Лагранжа (Малу теорему Ферма): $a^{-1} = a^{2^m-2}$.

4 Виконання роботи

Для реалізації скінченного поля $GF(2^{251})$ було розроблено клас `FieldElement251` де елемент зберігається у вигляді масиву з чотирьох 64-бітних цілих чисел (`std::array<uint64_t, 4>`), формуючи 256-бітний блок, з яких старші 5 бітів маскуються нулями.

З метою ознайомлення з прийомами ефективної реалізації було створено дві версії програми:

1. **Класична реалізація (`lab3_classic`):** Використовує базові алгоритми «зсув і додавання» для множення та ручне проріджування бітів для піднесення до квадрата у циклі.

2. **Апаратно-оптимізована реалізація (`lab3_opt`):** Використовує комп'ютерн інтринсики (compiler intrinsics) архітектури x86-64 та розширені алгоритми для максимальної швидкодії:

   - Інструкція `PCLMULQDQ` (`_mm_clmulepi64_si128`) для апаратного carry-less множення поліномів.
   - Інструкція `BMI2 PDEP` (`_pdep_u64`) для паралельного розкладання бітів при піднесенні до квадрата (за 1 такт замість циклу).
   - Інструкція `CLZ` (`__builtin_clzll`) для швидкого сканування нульових бітів під час редукції полінома степеня 500.
   - **Алгоритм Іто-Цудзіі (Itoh-Tsujii)** з використанням ланцюжків додавання для мінімізації кількості операцій множення при інверсії з 250 до 10.

4.1 Бітові зображення та контроль коректності

Обидві реалізації успішно проходять контроль коректності. Нижче наведено вивід програми, який підтверджує правильність обчислень та виконання теоретико-числових тотожностей:

```
--- OPERATIONS ---
Zero:       0000000000000000000000000000000000000000000000000000000000000000
One:        0000000000000000000000000000000000000000000000000000000000000001
A + B:      5975545B4AD6A4E539B9C666B8A5F2ABB8353DB255A4DD4266D7DF323D3FB83
A * B:      23FF11E372ACBCE3AD97D6E6A83CCCBD23AD2CE24511776C70616516DE5347D
A^2:        51048AB4CF41219DEE17D05E960737C521BA73BB4504211295029942 4D3C9A3
Trace(A):   1
A^-1:       32598E4C69BDBA2726611698DA60A94613AEAA82C42058E8669386236CF56D9
A * A^-1:   0000000000000000000000000000000000000000000000000000000000000001
Bin conv:   OK

--- IDENTITY CHECKS ---
(a + b) * c == b * c + c * a: PASSED
(a * b) * c == a * (b * c):   PASSED
a * b == b * a:               PASSED
a + a == 0:                   PASSED
a + 0 == a:                   PASSED
a * 1 == a:                   PASSED
a * 0 == 0:                   PASSED
a * a^-1 == 1:                PASSED
(a^-1)^-1 == a:               PASSED
(a + b)^2 == a^2 + b^2:       PASSED
a^2 == a * a:                 PASSED
Tr(a + b) == Tr(a) ^ Tr(b):   PASSED
Tr(a^2) == Tr(a):             PASSED
d^(2^m - 1) == 1:             PASSED
```

Результати виконання операцій та тотожностей

5 Аналітичні відомості та аналіз швидкодії (Бенчмарки)

Для визначення середнього часу виконання операцій було застосовано бібліотеку `std::chrono` (наносекунди) та інструкцію `__rdtsc()` (такти процесора). Результати вимірювань для обох варіантів реалізації зведено до таблиці 1.

Аналіз бенчмарків показує наступне:

- **Додавання:** Виконується на межі фізичних можливостей процесора (7-9 тактів), оскільки компілятор GCC (прапорець `-O3`) застосовує автовекторизацію (SIMD AVX2), виконуючи `XOR` усього 256-бітного масиву за одну інструкцію.

Таблиця 1: Порівняльний аналіз швидкодії (наносекунди та такти CPU)

Операція	**Класична реалізація**		**Апаратно-оптимізована**	
	Час (нс)	Такти	Час (нс)	Такти
Додавання (`+`)	1.28	9	1.69	7
Множення (`*`)	2620.56	8352	790.21	2518
Піднесення до квадрата	940.42	2997	341.49	1088
Слід елемента (`Trace`)	218655.00	696897	79250.40	252586
Обернений елемент	844795.00	2692533	127258.00	405596
Піднесення до степеня	528377.00	1684043	174251.00	555373

- **Множення:** Апаратна інструкція `PCLMULQDQ` забезпечила прискорення операції у ~ 3.3 раза (з 8352 до 2518 тактів) порівняно з програмними зсувами.
- **Квадрат:** Інструкція розкладання бітів `PDEP` прискорила операцію у ~ 2.7 раза, позбавивши необхідності обходити біти в циклі.
- **Обернений елемент:** Завдяки алгоритму Ітo-Цудзіі кількість важких операцій множення було зменшено з 250 до 10. У комбінації зі швидким апаратним квадратом це дало прискорення інверсії у ~ 6.6 раза (від 2.6 млн до 405 тис. тактів).

6 Висновки

Під час виконання лабораторної роботи було реалізовано скінченне поле Галуа $GF(2^{251})$ у поліноміальному базисі. Коректність математичного апарату підтверджена проходженням серії теоретико-числових тотожностей.

Важливим є дослідження методів ефективної реалізації криптографічних примітивів. Використання апаратних розширень сучасних процесорів (CLMUL, BMI2) через комп'ютерні інтринсики, а також імплементація просунутого алгоритму інверсії Ітo-Цудзіі дозволили прискорити критичні по часу ділянки коду в середньому від 3 до 7 разів порівняно з класичним алгоритмічним підходом. Це підкреслює важливість адаптації математичних алгоритмів під апаратну архітектуру при розробці продуктивних криптосистем.

Додаток А. Вихідний код оптимізованого ядра поля

Нижче наведено фрагмент реалізації апаратно-оптимізованих методів.

```
FieldElement251 FieldElement251::operator*(const FieldElement251& other) const {
    std::array<uint64_t, 8> wide; wide.fill(0);
    for (int i = 0; i < 4; ++i) {
        for (int j = 0; j < 4; ++j) {
            if (words[i] == 0 || other.words[j] == 0) continue;
            __m128i a = _mm_set_epi64x(0, words[i]);
            __m128i b = _mm_set_epi64x(0, other.words[j]);
            __m128i prod = _mm_clmulepi64_si128(a, b, 0x00);
            uint64_t p[2]; _mm_storeu_si128((__m128i*)p, prod);
            wide[i + j] ^= p[0]; wide[i + j + 1] ^= p[1];
        }
    }
    FieldElement251 res;
    reducePoly500(wide, res);
    return res;
}

FieldElement251 FieldElement251::square() const {
    std::array<uint64_t, 8> wide; wide.fill(0);
    for (size_t i = 0; i < NUM_WORDS; ++i) {
        wide[2 * i] = _pdep_u64(words[i], 0x5555555555555555ULL);
        wide[2 * i + 1] = _pdep_u64(words[i] >> 32, 0x5555555555555555ULL);
    }
    FieldElement251 res;
    reducePoly500(wide, res);
    return res;
}

static FieldElement251 power2k(const FieldElement251& a, int k) {
    FieldElement251 res = a;
    for (int i = 0; i < k; ++i) res = res.square();
    return res;
}

FieldElement251 FieldElement251::invert() const {
    if (isZero()) return zero();
    FieldElement251 S1 = *this;
    FieldElement251 S2 = S1 * power2k(S1, 1);
    FieldElement251 S3 = S2 * power2k(S1, 2);
    FieldElement251 S6 = S3 * power2k(S3, 3);
    FieldElement251 S12 = S6 * power2k(S6, 6);
    FieldElement251 S24 = S12 * power2k(S12, 12);
```

```
43      FieldElement251 S25 = S24 * power2k(S1, 24);
44      FieldElement251 S50 = S25 * power2k(S25, 25);
45      FieldElement251 S100 = S50 * power2k(S50, 50);
46      FieldElement251 S125 = S100 * power2k(S25, 100);
47      FieldElement251 S250 = S125 * power2k(S125, 125);
48      return S250.square();
49  }
```

Фрагмент FieldElement251_opt.cpp